Techniques web en python

Cours 3 : Web crawling et web scraping

Interview 3

Longest Palindromic Substring

Given a string s, return the longest palindromicsubstring in s.

NB ; un palindrome est un mot qui peut se lire dans les deux sens, comme CIC, Ressasser ...

```
[ ]   1 s1 = "iressasser"
      2 s2 = "babad"
      3 s3 =  "cbbd"
      4 s4 = "a"
```

Questions à se poser avant d’attaquer

- Comment décomposer le problème ? Peut-on isoler certaines instructions à répéter de nombreuses fois ?
- De combien de variables ai-je besoin ?
- Combien de fois je dois parcourir la chaîne de caractère ?

Se souvenir :

1) trouver une solution qui marche

2) trouver une solution efficace qui marche

3) trouver une solution efficace et élégante qui marche (optionnel)

1) Solution « bourrin »

```
]   1 def is_palindrome(substring):
    2
    3         for i in range(len(substring)) :
    4             if substring[i] != substring[-(i+1)] :
    5                 return False
    6
    7         return True
    8
    9 def longestPalindrome( s):
   10
   11     longest = 0
   12     longest_substring = ""
   13
   14     for i in range(len(s)+1) :
   15         for j in range(i,len(s)+1) :
   16             substring = s[i:j]
   17
   18             if is_palindrome(substring):
   19
   20                 if len(substring)>longest :
   21                   longest =  len(substring)
   22                   longest_substring = substring
   23
   24     print(longest_substring)
   25     return longest_substring
```

Solution 2 : optimiser la solution 1

```python
def is_palindrome(substring):
    #  Inutile de faire le test deux fois
    for i in range(len(substring) // 2):
        if substring[i] != substring[-(i+1)]:
            return False
    return True

def longestPalindrome(s):
    n = len(s)
    longest_substring = ""

    # ON parcourt toutes les substrings possibles
    for i in range(n):
        for j in range(i, n):
            substring = s[i:j+1]


            if is_palindrome(substring):
                if len(substring) > len(longest_substring):
                    longest_substring = substring

    print(longest_substring)
    return longest_substring
```

Solution 3 : repenser le problème autrement

```
expand_around_center(s, left, right):
        # renvoie le plus long palindrome à partir d un caractere , O(n)
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return s[left + 1:right]


longestPalindrome( s):

    if not s:
        return ""

    longest_substring = ""

    for i in range(len(s)):
        # distinction des cas pairs et impairs
        odd_palindrome = expand_around_center(s,i, i)
        even_palindrome = expand_around_center(s,i, i + 1)

        longest_substring = max(longest_substring, odd_palindrome, even_palindrome, key=len

    print(longest_substring)
    return longest_substring
```

De quoi est constitué le web

Rappel : internet = réseau de machines capables d'échanger des informations

Web : Le World Wide Web, abrégé en WWW, W3, le Web, la toile mondiale ou simplement la toile, est un système hypertexte public fonctionnant sur Internet

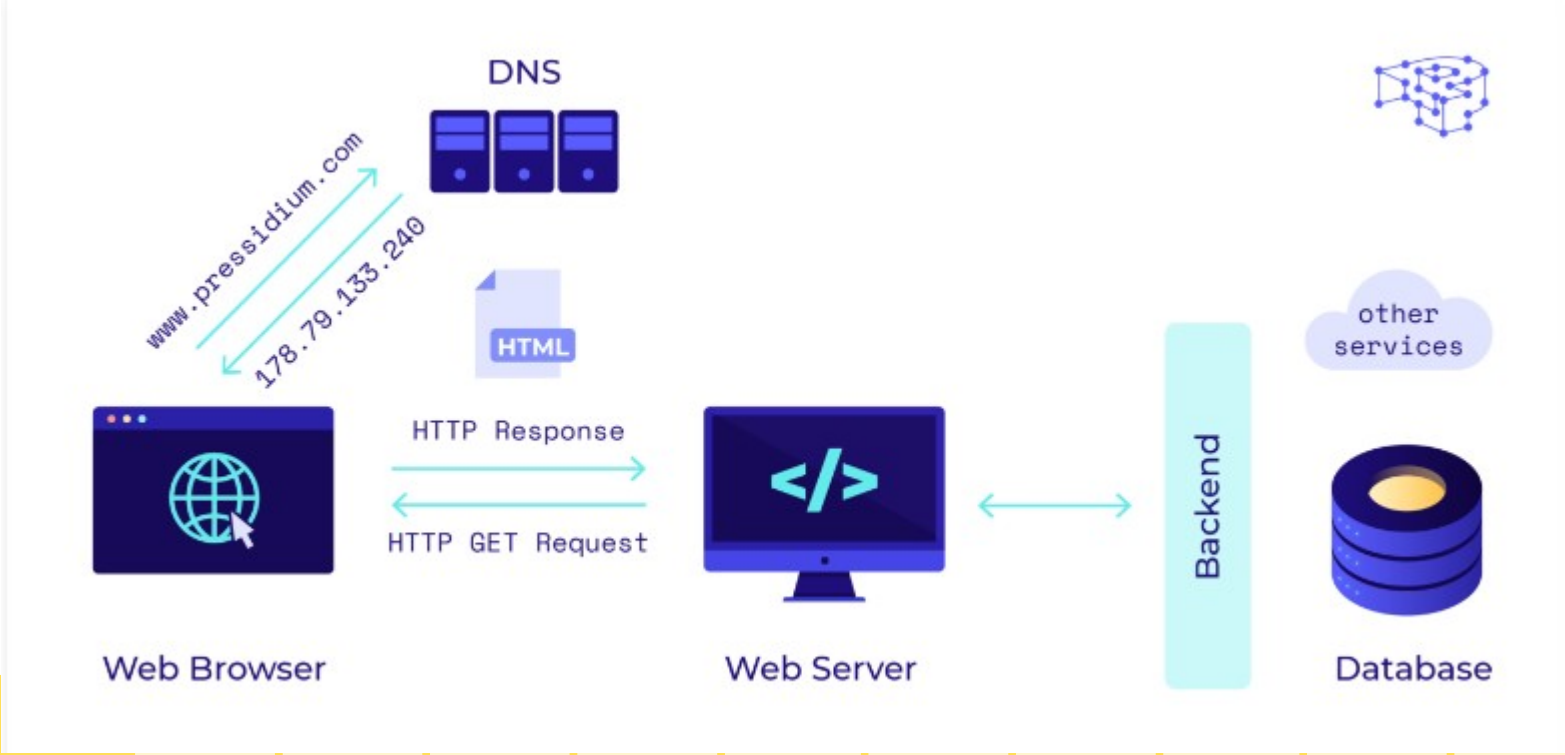
=> une application qui fonctionne grâce à internet.

- Des serveurs stockent des pages.
- Par le biais d'une application (le web browser, navigateur web), on peut explorer ces pages.

Que se passe t’il lorsque je veux accéder à un site ?

https://www.google.com/search?sca_esv=f5c9479add9f09fd&sxsrf=AE3TifNgGYLqQfPBTMksb
ect+to+a+website&sou
/K3UOyxRODOGo5r7
sGkOilAtE8mDAnlgi6
J1kFbAwewXek00mh
CRAB&biw=2480&bih=

n-you-visit-a-url/



Les types de requête

📡 Les types de requêtes HTTP

- **GET** :
  - Récupérer une ressource (page, image, API…)
  - Lecture seule → ne modifie rien
  - Exemple : consulter `http://example.com/index.html`
- **POST** :
  - Envoyer des données au serveur
  - Utilisé pour **créer** une ressource
  - Exemple : formulaire de connexion, envoi de commentaire
- **PUT** :
  - **Remplacer** une ressource existante par une nouvelle
  - Idempotent (même requête → même résultat)
- **PATCH** :
  - **Modifier partiellement** une ressource
  - Plus léger que PUT
- **DELETE** :
  - Supprimer une ressource

- Au départ que get,
- Puis besoin d’ajouter de nouvelles requêtes avec l’évolution du web et l’ajout d’un dynamisme (capacité de l utilisateur à modifier le site / la page affichée)

Pourquoi HTTP

- HT HyperText

  Du texte enrichi avec des informations sur son placement, son organisation, des liens…

  Fait partie du contrat initial pensé par les créateurs d’internet back in the days…

  A finalement peu changé

  L’hypertexte à un langage : le HTML !

HTML : le langage du web

🧱 HTML : le langage du Web

- **Signification** : *HyperText Markup Language*
- **Créateur** : **Tim Berners-Lee**, en **1991** au **CERN**
- **Rôle** : décrire la **structure et le contenu** d'une page Web
- **Principe** : un système de **balises** (*tags*) qui encadrent les éléments

```html
<h1>Bienvenue sur le Web</h1>
<p>Ceci est un paragraphe.</p>
<a href="https://www.example.com">Un lien</a>
```

- **Évolution** :
  - HTML 1.0 → 1991 : structure de base
  - HTML 4.01 → 1999 : standardisation W3C
  - **HTML5 → 2014** : multimédia, vidéo, canvas, API modernes
- **Pourquoi important ?**
  - Base de tout site web (avec CSS + JS)
  - Lisible par les navigateurs et les moteurs de recherche
  - Permet l'**hypertexte**, fondement même du Web 🌐

Langage purement utilisé pour la structuration de contenu !!!

Quelles sont les balises importantes ?

- https://www.geeksforgeeks.org/html/most-commonly-used-tags-

Document structure tag

The <html> tag is essential, encapsulating the entire HTML document and serving as the root element for content organization.

Structure Tag	Syntax	Description
<html>	<html> Statements... </html>	The root of the HTML document specifies it as HTML.
<head>	<head> Statements... </head>	Contains head elements such as title, style, and meta tags in the HTML file.
<title>	<title> Statements... </title>	Defines the title of an HTML document.
<body>	<body> Statements... </body>	Defines the body of an HTML document, containing content like images, tables, and lists.

Commonly used tags in HTML

HTML Tag	Syntax	Description
	...	Defines a division or section in an HTML document.
	...	Defines a paragraph.
<a>	...	Defines a hyperlink.
<img>	<img src="..." alt="...">	Embeds an image.
	...	Defines an unordered list.
	...	Defines an ordered list.
	...	Defines a list item.
<table>	<table>...</table>	Defines a table.
<tr>	<tr>...</tr>	Defines a table row.
<th>	<th>...</th>	Defines a table header cell.
<td>	<td>...</td>	Defines a table data cell.
<form>	<form>...</form>	Defines an HTML form for user input.
<input>	<input type="...">	Defines an input control within a form.
<button>	<button>...</button>	Defines a clickable button.
<h1>-<h6>	...	Define headings of different levels.
	...	Defines a generic inline container.
<label>	<label for="...">...</label>	Defines a label for an input element.
<iframe>	<iframe src="..."></iframe>	Embeds an inline frame for external content.

Essayez de grouper ces balises,

- A quoi servent-elles ?

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>The Best INALCO Class Ever</title>
</head>
<body>
  <h1>The Best INALCO Class Ever 🎓</h1>

  <h2>Web Techniques (TALA526A)</h2>
  <p>
    Taught by <strong>Louis Jourdain</strong> — a journey from the <em>Minitel</em> to modern AI-powered web systems.
  </p>

  <h3>Why it’s the best</h3>
  <ul>
    <li>Explains how the Web was born 🌍</li>
    <li>Teaches how the Internet really works ⚙️</li>
    <li>Combines code, history, and creativity 💡</li>
  </ul>

  <h3>Example of what we learn</h3>
  <p>Understanding a simple HTTP request:</p>
  <pre>
GET /index.html HTTP/1.1
Host: www.inalco.fr
  </pre>

  <p>💬 “In this class, the Web finally makes sense.”</p>
</body>
</html>
```

The Best INALCO Class Ever 🎓

Web Techniques (TALA526A)

Taught by **Louis Jourdain** — a journey from the *Minitel* to modern AI-powered web systems.

Why it's the best

- Explains how the Web was born 🌍
- Teaches how the Internet really works ⚙️
- Combines code, history, and creativity 💡

Example of what we learn

Understanding a simple HTTP request:

vbnet Copier le code

```
GET /index.html HTTP/1.1
Host: www.inalco.fr
```

💬 "In this class, the Web finally makes sense."

The Best INALCO Class Ever 🎓

Web Techniques (TALA526A)

Taught by **Louis Jourdain** — a journey from the *Minitel* to modern AI-powered web systems.

Why it's the best

- Explains how the Web was born 🌍
- Teaches how the Internet really works ⚙️
- Combines code, history, and creativity 💡

Example of what we learn

Understanding a simple HTTP request:

vbnet Copier le code

```
GET /index.html HTTP/1.1
Host: www.inalco.fr
```

💬 "In this class, the Web finally makes sense."

Un peu de couleur dans tout ça !

🎨 CSS : Cascading Style Sheets

Date	Événement clé	Description
1994	💡 *Invention*	Proposé par **Håkon Wium Lie** au **CERN** → séparer le **contenu (HTML)** de la **mise en forme (style)**
1996	📜 *Standardisation*	**CSS1** publié par le **W3C** : couleurs, polices, marges, alignements
1998	🧱 *CSS2*	Positionnement, z-index, styles selon le média (écran, impression)
1999–2004	⚔️ *Browser Wars*	Incompatibilités entre navigateurs (IE, Netscape) → "CSS hacks"
2011 → aujourd'hui	🚀 *CSS3 et +*	Modules modernes : **Flexbox**, **Grid**, **Animations**, **Variables**

Un décodage pour l’affichage

- CSS est stocké dans un fichier à part lié à la page.
- Il décrit comment les éléments balisés par chaque balise doivent être affichés.
- Le navigateur web :

  Charge le HTML

  Charge le CSS

  Calcule quelles règles s’appliquent à chaque élément

  Affiche la page stylisée

```css
/* ====== The Best INALCO Class Ever ====== */
/* Basic page styling for readability and clarity */

body {
  font-family: "Segoe UI", Arial, sans-serif;
  background-color: #f8f9fa;
  color: #222;
  margin: 40px;
  line-height: 1.6;
}

h1 {
  color: #0056b3;
  text-align: center;
}

h2 {
  color: #0069d9;
  text-align: center;
  margin-top: -10px;
}

h3 {
  color: #333;
  margin-top: 25px;
}

p {
  max-width: 600px;
  margin: 0 auto 1em;
  text-align: center;
}
```

```css
ul {
  list-style-type: "💡 ";
  max-width: 500px;
  margin: 0 auto;
  padding-left: 20px;
}

pre {
  background-color: #eee;
  border-left: 4px solid #0056b3;
  padding: 10px;
  max-width: 450px;
  margin: 10px auto;
  font-family: "Courier New", monospace;
  font-size: 0.9em;
}

strong {
  color: #d63384;
}

em {
  color: #198754;
  font-style: normal;
  font-weight: bold;
}

footer {
  text-align: center;
  margin-top: 40px;
  font-size: 0.9em;
  color: #555;
}
```

The Best INALCO Class Ever

Web Techniques (TALA526A)

Taught by Louis Jourdain — a journey from the Minitel to modern AI-powered web systems.

Why it's the best

- Explains how the Web was born 🌍
- Teaches how the Internet really works ⚙️
- Combines code, history, and creativity 💡

Example of what we learn

```
GET /index.html HTTP/1.1
Host: www.inalco.fr
```

💬 *"In this class, the Web finally makes sense."*

- C'est mieux avec couleur non ?

- Mais trouvez-vous ça joli ?
- Quelles sont les limitates ?
- Puis-je me créer un compte pour le cours ?

⚡ JavaScript — The Brain of the Web

Aspect	Description
🧩 **What it is**	A **programming language** created in **1995** by **Brendan Eich** at Netscape to make web pages **interactive and dynamic**.
🧠 **Role in the Web trio**	**HTML** → structure • **CSS** → style • **JavaScript** → behavior
⚙️ **How it works**	Runs directly inside the **browser's engine** (e.g. V8 in Chrome). Scripts can modify, add, or remove HTML elements in real time.
💡 **Example**	`html <button onclick="alert('Hello!')">Click me</button>` → shows a pop-up when clicked.
🌐 **Why it matters**	Enables interactivity (menus, forms, animations), communicates with servers (APIs), and powers web apps.
🧱 **Beyond the browser**	Thanks to **Node.js**, JavaScript also runs on the **server side**, building full-stack applications.

In short:

> 🏗️ **HTML** builds • 🎨 **CSS** styles • ⚡ **JavaScript** brings life

- Le navigateur reçoit du HTML
- Le html contient du lien vers d'autres fichiers (autre pages html, css, ou aussi javascript qui sont chargées par d'autres requêtes GET)
- Le code javascript est exécuté dans votre navigateur. Ce code transforme dynamiquement la page qui est affichée

🧩 DOM — The Document Object Model

Aspect	Explanation
🧠 **Definition**	The **DOM** is a *tree-like structure* representing everything in an HTML or XML document.
📄 **Purpose**	It allows **JavaScript to access and modify** the structure, content, and style of a web page.
🌳 **Structure**	Every element (like `<body>`, `<p>`, `<img>`) becomes a **node** in this tree. Text, attributes, and even comments are nodes too.
⚙️ **How it works**	When the browser loads an HTML file, it parses it and builds the **DOM tree in memory**. JavaScript can then read or change any node.
💡 **Example**	`js document.getElementById("title").style.color = "blue";` → instantly changes the color of the element with id *title*.
🔄 **Dynamic updates**	Changes made by JavaScript **update the visible page immediately**, without reloading from the server.

🧠 In short

> The **DOM** is the *living model* of your web page inside the browser.
> HTML builds it, CSS styles it, JavaScript manipulates it.

Un écosystème en pleine évolution

⚙️ From Static Pages to Modern Web Frameworks

Era	Technology	How It Works	What It Brought
🕰️ **1990s**	**HTML + CSS + JS**	Browser loads static files from the server	Simple, fast, no interactivity beyond basic scripts
⚡ **2000s**	**AJAX + jQuery**	JS fetches data *without reloading* the page	Dynamic pages, early web apps (Gmail, Google Maps)
🧱 **2010s**	**Front-end frameworks** (React, Vue, Angular)	The **browser** builds the interface dynamically using components	Apps feel like native software (SPAs)
🚀 **Now**	**Full-stack JS** (Node.js, Next.js, Remix…)	Same language (JavaScript) used **front + back**	Real-time, server-rendered, API-driven web apps

Web crawling et web scraping

Web crawling : indexation

opération effectuée par des programmes (crawlers) des moteurs de recherche pour capturer l'essentiel d'une page web dans le but d'indexer le site. => volume mais faible précision

Web scaping : extraction de données web

On connaît exactement l'url de la page qui nous intéresse

scraper : extraction spécifique par traitement d'une page html

https://brightdata.fr/blog/leadership-fr/web-crawling-vs-web-scraping

Crawling et scraping :

Des opérations complémentaires :

crawling : récupérer toutes les url en temps réel et une idée de leur contenu (On en fera au TP 3 😈)

scraping : à partir d'une url analyser le html pour en retirer l'information pertinente

L’objectif du scraping : collecter la donnée de valeur

MAIS :

- Internet est pollué par les contenus (pub, renvois…) non pertinents
- Chaque site est unique et peut présenter des balises HTML différentes => pas de script unique capable de scraper le contenu qualitatif de n’importe quelle page.
- Attention aux questions de droit d’accès : consulter le fichier robot.txt qui est au départ de l’architecture d’un site

POLITIQUE • LES RÉPUBLICAINS (LR)

Eric Ciotti annonce quitter Les Républicains, après plusieurs mois de bataille juridique pour rester à sa présidence

Celui qui était encore président du parti de droite, contesté depuis son alliance avec le Rassemblement national, dit, dans un entretien au « Figaro », constater « la dissolution de l'état-major des Républicains dans la Macronie ».

Le Monde
Publié aujourd'hui à 16h51, modifié à 17h46 · Lecture 2 min.

Offrir l'article



Les plus lus

1 Eric Ciotti annonce quitter Les Républicains, après plusieurs mois de bataille juridique pour rester à sa présidence

Se connecter S'abonner

once quitter Les
près plusieurs mois de
ue pour rester à sa

ident du parti de droite, contesté depuis son
ment national, dit, dans un entretien au « Figaro »,
e l'état-major des Républicains dans la Macronie ».

17h46 · Lecture 2 min.



DevTools is now available in French! Always match Chrome's language Switch DevTools to French

Elements Console Sources Network Performance Memory Application

```
<div class="Header__nav-container">…</div>
<div class="lmd-banners js-lmd-banners"> </div>
<main role="main" class="main main LeMondeMain  main--free  ">
  <div id="dfp-habillage" class="dfp-slot dfp__slot dfp__slot--habillage" data-format="habillage" aria-hidden="true"></div>
  <div class="dfp__container">  flex  == $0
    <div id="banniere_haute" class="dfp-slot dfp__slot dfp__slot--banniere dfp__slot--banniere-haute adm-ad-loading" data-format="banniere_haute" aria-hidden="true">…</div>
  </div>
  <div id="js-modal-gifted-url" class="article__gift-modal">…</div>
  <section id="habillagepub" class="article article--single article--iso article--content ">
    <section class="zone zone--article  old__zone">
      <header class="article__header  ">
        <div class="article__header-wrap">
          <ul class="breadcrumb">  flex
            <li class="breadcrumb__parent breadcrumb__parent--after js-breadcrumb">…</li>  flex
            <li class="breadcrumb__child--article js-breadcrumb">…</li>  flex
          </ul>
          <div class="article__heading">
            <h1 class="article__title">
              "Eric Ciotti annonce quitter Les Républicains, après plusieurs mois de bataille juridique pour rester à sa présidence"
            </h1>
            <p class="article__desc">
              " Celui qui était encore président du parti de droite, contesté depuis son alliance avec le Rassemblement national, dit, dans un entretien au « Figaro », constater « la dissolution de l'état-major des Républicains dans la Macronie ». "
            </p>
          </div>
          <p class="meta meta__publisher meta__publisher--header" id="js-authors-trigger">  flex
            <span class="meta__author meta__author--no-after">Le Monde</span>
          </p>
          <section class="meta__date-reading">…</section>
          <div class="share-btns meta meta__social  ">…</div>  flex
        </div>
        ::after
```

onts-loaded body#js-body.js-dom-ready main.main.LeMondeMain.main--free. div.dfp__container

Console Search

Les étapes du scraping réussi

1) Avoir le bon url de la page qu'on va scanner

=> charger le contenu HTML de la page

2) déterminer derrière quelle.s balise.s se cachent les informations qui nous intéressent

=> ne sélectionner que l'information contenue dans ces balises

3) stocker l'information où on veut

=> récupérer l'information dans une variable qui nous intéressent pour la suite de notre aventure algorithmique.

Les frameworks python pour faire du scraping

Toutes les étapes du scraping réussi sont prises en charge par des librairies spécialisées.

- Le faire soi même avec urlib + html (conserve la structure) et coder une fonction avec des expressions régulières

  pattern = "<title.*?>.*?</title.*?>" #contenu d'une balise

  match_results = re.search(pattern, html, re.IGNORECASE)
- Utiliser un parser HTML d'une librairie spécialisée

  (Beautifulsoup, trafilatura, scrapy...)
- https://realpython.com/python-web-scraping-practical-introduction/

1) chargement du contenu HTML

- Utilisation des librairies urllib / requests

```
url = ‘blabla.com’
headers = { 'User-agent' : 'computer science student mail' }

#création d’un objet requête
request = urllib.request.Request(url, headers=headers)

# récupération du contenu (string)
with urllib.request.urlopen(request) as webpage:
    html = webpage.read()
```

2) parsing de la chaîne

Installer beautiful soup pip install beautifulsoup4

- C créer un parser HTML de la page

  soup = BeautifulSoup(html)
- Trouver les sections de pages correspondant à une balise en particulier

  anchors = soup.find_all('balise')
- récupérer les textes et en faire ce qu’on veut ...

  for anchor in anchors:

  texte = anchor.get_text()

2) parsing de la chaîne

Utilisez les avantages de la librairie (el.descendants, el.children…) pour analyser au mieux la structure du fichier HTML qu'on considère

Il faut souvent expérimenter avant de trouver la bonne formule !